EDR7 Module Integration Spec

Oct 14, 2026 · @.

Purpose

Every module in our EDR plugs into one shared detection engine, so we all build to the same three contracts: the **Event**, the **Rule** and the **Alert**. If your module follows this spec, it will run inside `mini_edr.py` on demo day with no rework.

The core rule: **agents only collect, the engine detects, the responder acts.** This is a change from the starter code, where each agent did its own `detect()` and `respond()`. Under this spec, agents return Events, detection logic lives as rules in `rules/*.yaml`, and every response goes through one Responder.

Why we are doing it this way:

- The client asked for a customisable EDR. Rules in a YAML file can be added or tuned without touching code.
- Every alert gets the same MITRE tag and the same MTTD timing, which is our evaluation focus.
- A bug in one module cannot break another module or crash the EDR.

Questions about this spec go to David as comments on this doc.

Schedule to Friday

The presentation is **Friday 23 October at 4:30pm**, so code freezes at the end of Wednesday 21 October. Please start your part before Monday's meeting, not after it.

When	Goal	Who
Before Monday	Draft your rule file and your test cases. Louis starts `agent_fim.py`. David finishes the engine, responder and RCE agent.	Everyone
Mon 19 Oct, 12pm	Meeting: confirm this schedule, decide XSS, split the slides, answer open questions	Everyone

When	Goal	Who
Tue 20 Oct	First full integration run with every rule file loaded together; fix clashes	David + each slice owner
Wed 21 Oct	Full scoring run and Wazuh comparison on the same cases; pick the demo attacks; code freeze at end of day	David runs it, everyone checks their slice
Wed 21 Oct evening	Results go into the slides	Slide owners
Thu 22 Oct	Slides final; two full rehearsals; record a backup demo video	Everyone
Fri 23 Oct, 4:30pm	Presentation	Everyone

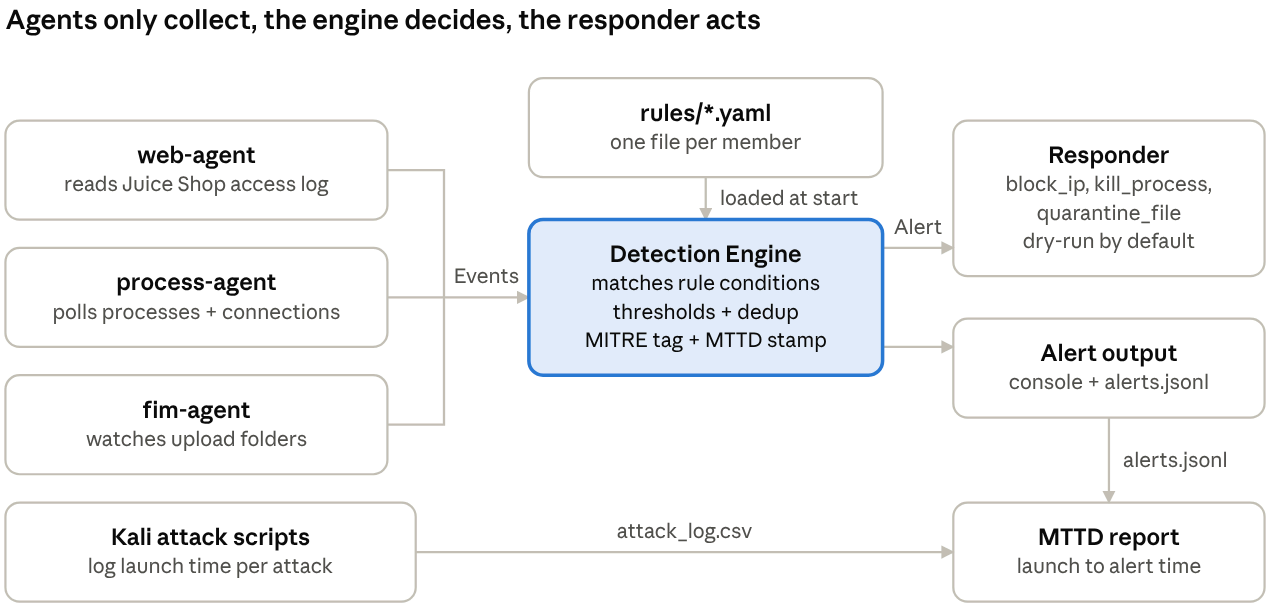
Must have by Wednesday:

- Each slice's rules detect its malicious test cases
- The TP / FP / TN / FN scoring runs end to end
- MTTD compared against the Wazuh baseline

Nice to have: live response actions (`--respond`) beyond the one showcase in the demo.

If a slice is not working by Wednesday, it goes in the results as a false negative and we explain why in the slides. That is honest, still scores well, and is far better than a broken live demo.

Architecture



EDR data flow · 3 agents, 1 engine, 1 responder

Each agent turns raw telemetry into Events. The engine checks every Event against everyone's rule files, then hands each Alert to the Responder and the output log, where the MTTD report picks it up alongside the Kali launch times.

UML class diagram

Three agents inherit from one abstract `Agent`, and everything else is shared code that David builds. You only need to understand `Agent`, `Event` and `Rule` to write your part.

```
classDiagram
    direction LR
    class MiniEDR {
        +agents: list~Agent~
        +engine: DetectionEngine
        +responder: Responder
        +sink: AlertSink
        +run()
    }
    class Agent {
        <<abstract>>
        +name: str
        +interval: float
```

```
        +collect() list~Event~
    }
    class WebAgent {
        +log_path: str
    }
    class ProcessAgent {
        +seen_pids: set
    }
    class FimAgent {
        +watch_dirs: list
        +known_hashes: dict
    }
    class Event {
        +event_id: str
        +agent: str
        +event_type: str
        +observed_at: float
        +collected_at: float
        +host: str
        +data: dict
    }
    class Rule {
        +id: str
        +title: str
        +owner: str
        +event_type: str
        +severity: str
        +mitre: str
        +conditions: list
        +threshold: dict
        +response: str
        +matches(event) bool
    }
    class DetectionEngine {
        +rules: list~Rule~
        +windows: dict
        +load_rules(folder)
        +evaluate(events) list~Alert~
    }
    class Alert {
        +alert_id: str
        +rule_id: str
        +title: str
```

```
        +severity: str
        +mitre: str
        +event_ids: list
        +observed_at: float
        +detected_at: float
        +mttd_s: float
        +details: dict
        +response: dict
    }
    class Responder {
        +respond_enabled: bool
        +handle(alert)
        -block_ip(ip)
        -kill_process(pid)
        -quarantine_file(path)
    }
    class AlertSink {
        +path: str
        +write(alert)
    }
    Agent <|-- WebAgent
    Agent <|-- ProcessAgent
    Agent <|-- FimAgent
    MiniEDR o-- "1..*" Agent
    MiniEDR *-- DetectionEngine
    MiniEDR *-- Responder
    MiniEDR *-- AlertSink
    Agent ..> Event : creates
    DetectionEngine "1" *-- "*" Rule : loads
    DetectionEngine ..> Event : evaluates
    DetectionEngine ..> Alert : creates
    Responder ..> Alert : acts on
    AlertSink ..> Alert : writes
```

File	Classes	Owner
`edr_core.py`	`Event`, `Alert`, `Agent`, `AlertSink`	David
`engine.py`	`Rule`, `DetectionEngine`	David
`responder.py`	`Responder`	David
`mini_edr.py`	`MiniEDR`	David

File	Classes	Owner
`agent_web.py`	`WebAgent`	David
`agent_process.py`	`ProcessAgent`	David
`agent_fim.py`	`FimAgent`	Louis
`rules/*.yaml`	one rule file per member	everyone

Event schema

An Event is one thing that happened on the Juice Shop VM. Your agent's `collect()` returns a list of these and nothing else.

```
@dataclass
class Event:
    agent: str            # e.g. "web-agent"
    event_type: str       # one of the four types below
    observed_at: float    # epoch seconds, when it HAPPENED (from the source)
    data: dict            # type-specific fields, see table below
    host: str = field(default_factory=socket.gethostname)
    collected_at: float = field(default_factory=time.time)
    event_id: str = field(default_factory=lambda: str(uuid.uuid4()))
```

`observed_at` must come from the source, never from `time.time()`. Use the log line's timestamp, the process `create_time()`, or the file's `mtime`. If you use poll time, our MTTD numbers are wrong.

event_type	Collected by	Required `data` fields
`http_request`	web-agent	`ip`, `method`, `path` (URL-decoded twice with `unquote`), `raw_path`, `status` (int), `user_agent`
`process_start`	process-agent	`pid`, `ppid`, `name`, `parent_name`, `cmdline`, `user`
`file_change`	fim-agent	`path`, `change` (`created` / `modified` / `deleted`), `sha256`, `size`
`net_connection`	process-agent	`pid`, `process_name`, `laddr`, `raddr`, `rport`, `status`

`process_start` events come from auditd's `audit.log` (the `proc_exec` rule on the Juice Shop user), not from polling with `psutil`. RCE commands like `id` or `whoami` finish in milliseconds, so a 2 second poll misses them. auditd records every one, and it is the same evidence Wazuh uses, which keeps our comparison fair.

For `file_change`, a `moved` event from watchdog is reported as `created` with the destination path.

Use these exact field names, because rules match on them. Need a new field or event type? Comment here first so it gets added to this table for everyone.

Known limitation: the Juice Shop access log records URLs only, never POST bodies. Attacks sent in a request body (login SQLi, B2B order RCE) will not appear in `http_request` events, so plan your detection around their side effects (status codes, processes, files).

Rule format

Your detection logic is written as rules in `rules/*.yaml`, not as Python. A rule fires when **all** of its conditions match one Event of its `event_type`.

```
- id: WEB-SQLI-001
  title: SQL injection in product search
  owner: Thapanawat
  event_type: http_request
  severity: HIGH                # LOW | MEDIUM | HIGH
  mitre: T1190                  # ATT&CK technique ID
  conditions:                   # all must match
    - {field: path, op: regex, value: "/rest/products/search"}
    - {field: path, op: regex, value: "union\\s+select|sleep\\("}
  response: block_ip
```

Count-based attacks add a `threshold`. The window is measured on `observed_at`, so it works the same however fast the agents poll:

```
- id: WEB-BRUTE-001
  title: Brute-force login
  owner: Oliver
  event_type: http_request
  severity: HIGH
  mitre: T1110.001
  conditions:
    - {field: path, op: regex, value: "/rest/user/login"}
    - {field: status, op: in, value: [401]}
  threshold: {count: 5, window_s: 15, group_by: ip}
  response: block_ip
```

Process-based example:

```
- id: PROC-RCE-001
  title: Shell spawned by the Juice Shop process
  owner: David
  event_type: process_start
  severity: HIGH
  mitre: T1059.004
  conditions:
    - {field: parent_name, op: equals, value: node}
    - {field: name, op: in, value: [sh, bash, nc, curl, wget, python3]}
  response: kill_process
```

Operator	Matches when	Example `value`
`equals`	field equals value exactly	`node`
`in`	field is one of a list	`[401, 403]`
`regex`	case-insensitive regex search finds a match	`"union\\s+select"`
`contains`	value is a substring of the field	`/ftp/`
`gt` / `lt`	number is greater / less than value	`1048576`

Response	Acts on	What it does
`block_ip`	`data.ip`	iptables DROP from that IP
`kill_process`	`data.pid`	kills the process

Response	Acts on	What it does
`quarantine_file`	`data.path`	moves the file to `~/edr_lab/quarantine`
`none`	nothing	alert only

Rule IDs follow `SOURCE-ATTACK-NNN` (`WEB`, `PROC`, `FILE`, `NET`), for example `FILE-UPLOAD-002`.

Alert format and MTTD

The engine builds every Alert. You never create one yourself, but you will read them in `~/edr_lab/alerts.jsonl` (one JSON object per line) when testing your rules.

Field	Type	Meaning
`alert_id`	str	unique ID
`rule_id`	str	the rule that fired, e.g. `PROC-RCE-001`
`title`, `severity`, `mitre`	str	copied from the rule
`agent`	str	agent that collected the triggering Event
`event_ids`	list[str]	every Event that caused it (several for a threshold rule)
`observed_at`	float	`observed_at` of the first triggering Event
`detected_at`	float	when the engine raised the alert
`mttd_s`	float	`detected_at - observed_at`
`details`	dict	the triggering Event's `data`
`response`	dict	action name and outcome: `performed`, `dry-run` or `failed`

```
{"alert_id": "a41c...", "rule_id": "WEB-BRUTE-001", "title": "Brute-force
login", "severity": "HIGH", "mitre": "T1110.001", "agent": "web-agent",
"event_ids": ["..."], "observed_at": 1760425800.12, "detected_at":
1760425802.47, "mttd_s": 2.35, "details": {"ip": "192.168.56.20", "path":
"/rest/user/login", "status": 401}, "response": {"action": "block_ip",
"outcome": "dry-run"}}
```

We report MTTD two ways:

1. **Pipeline MTTD** = `mttd_s` above: how long our EDR takes once the evidence exists on the host.
2. **End-to-end MTTD** = time from attack launch to alert. Every attack script on Kali appends `{case_id, label, expected_rule, launched_at}` to `attack_log.csv`. After a run, a script matches each attack to its first alert and computes `detected_at - launched_at`. The same playbook run against the Wazuh agent gives our baseline.

End-to-end MTTD only works if both VMs agree on the time. Run `date` on both before each test run and note any offset.

Testing detection accuracy

Every rule is scored against labelled test cases, so the demo and final report show true and false positives and negatives for each slice, not just "it caught the attack".

	Alert raised	No alert
Malicious case (an attack)	**True positive (TP):** attack caught	**False negative (FN):** attack missed
Benign case (normal use)	**False positive (FP):** false alarm	**True negative (TN):** correctly quiet

Test cases

Each member adds two kinds of case for their slice to `tests/testcases.yaml`:

- **Malicious variants (at least 3):** the attack in different forms (encodings, payloads, speeds), to find what slips through as FNs.

- **Benign look-alikes (at least 3):** normal use that looks close to the attack, to find FPs. For example: searching for "O'Brien" (apostrophe vs the SQLi rule), four wrong passwords then the right one (vs the brute-force threshold), uploading a real PDF (vs the upload rule), a normal login (vs the auth rule).

```
- id: TC-RCE-01
  owner: David
  label: malicious
  expected_rule: PROC-RCE-001
  command: python3 attacks/rce.py --payload reverse-shell

- id: TC-RCE-B01
  owner: David
  label: benign
  expected_rule: none
  command: python3 benign/browse.py --pages 20
```

How a run is scored

1. Run the EDR in **dry-run mode**. With responses on, the first IP block stops every later case and turns them all into false negatives.
2. `run_eval.py` on Kali runs the cases one at a time. For each it appends `{case_id, label, expected_rule, launched_at}` to `attack_log.csv`, then waits longer than any rule's threshold window (20 seconds) so alerts can be matched by time. This is our existing attack runner with two columns added to its CSV: label and expected_rule.
3. `score_eval.py` reads `attack_log.csv` and `alerts.jsonl` and labels each case:

Case label	Alerts in that case's window	Result
malicious	at least one from `expected_rule`	TP (repeat alerts count once)
malicious	none from `expected_rule`	FN
malicious	an alert from a different rule	also one FP (misclassified)
benign	any alert	FP
benign	none	TN
no case running	any alert	FP (stray alert)

4. It prints these metrics per slice and overall:

Metric	Formula	Tells us
Detection rate (recall)	TP / (TP + FN)	share of attacks caught
Precision	TP / (TP + FP)	share of alerts that were real
False positive rate	FP / (FP + TN)	how often normal use triggers an alarm
Accuracy	(TP + TN) / all cases	overall correctness
F1 score	2 × precision × recall / (precision + recall)	one score balancing both
MTTD	mean of `detected_at - launched_at` over TPs	speed, our main focus

Run the same suite against the Wazuh baseline so both are scored on identical cases. Wazuh has its own rule IDs, so match its alerts by time window and MITRE technique instead of `expected_rule`.

Who owns what

Each member owns one attack slice end to end: the attack script, the rules that detect it, and any agent it needs. Each person writes rules in their **own file** in `rules/` (the engine loads every file there), so nobody edits the same file and there are no merge conflicts.

Member	Attack slice	Event types used	Agent work	Suggested MITRE	Rule file / ID prefix
David	RCE + engine, responder, demo	`process_start`, `net_connection`	writes `agent_process.py`, fixes `agent_web.py`	T1190, T1059.004	`rules/rce.yaml` / `PROC-RCE`
Louis	Malicious file upload	`file_change`, `http_request`	writes `agent_fim.py` (watches the Juice Shop upload folders)	T1505.003	`rules/upload.yaml` / `FILE-UPLOAD`
Thapanawat	SQL injection	`http_request`	none, uses web-agent	T1190	`rules/sqli.yaml` / `WEB-SQLI`

Member	Attack slice	Event types used	Agent work	Suggested MITRE	Rule file / ID prefix
Kikiola	Broken authentication	`http_request`	none, uses web-agent	T1078	`rules/auth.yaml` / `WEB-AUTH`
Oliver	Brute force	`http_request`	none, uses web-agent	T1110.001	`rules/brute.yaml` / `WEB-BRUTE`

XSS is no longer assigned. We decide at Monday's meeting whether someone takes it on or it is dropped from scope.

The MITRE IDs are a starting point. Each owner confirms the technique that best fits their attack and notes why in their section of the final report.

If your attack does not show up in any Event type above (for example a payload hidden in a POST body), tell David early. We either add telemetry or detect a side effect instead.

How to run it

On the Juice Shop VM:

```
pip3 install psutil pyyaml watchdog
cd ~/edr                        # the EDR code from our shared folder
sudo python3 mini_edr.py         # dry-run: alerts only, nothing is blocked
```

`sudo` is needed to read `audit.log`. Dry-run is the default; do not add `--respond` unless David asks.

To test your slice:

1. Start the EDR on the Juice Shop VM as above.
2. On Kali, run your attack script.
3. Watch the EDR console for your alert, or run `tail -f ~/edr_lab/alerts.jsonl` on the Juice Shop VM.
4. No alert? Check the console for rule errors at startup, then check your rule's `event_type` and field names against the Event schema table.
5. Run your benign test cases too and confirm they raise no alerts.

Stuck for more than 30 minutes? Comment on this doc or message David with the console output.

Coding rules and definition of done

If you are writing an agent, follow these rules.

1. Subclass `Agent` from `edr_core.py` and implement `collect()` only. It returns a `list[Event]`, empty if nothing new happened.
2. `collect()` must never raise. Wrap it in `try/except`, print the error, and return `[]`.
3. `collect()` must return fast, well under the 2 second interval. No `sleep()`, no blocking reads.
4. Report only new activity since the last call: seek to the end of logs, and remember PIDs or file hashes already seen.
5. Agents have no side effects. They never kill, block, move or delete anything; that is the Responder's job.
6. Only run `--respond` against the lab VMs. Default is dry-run.
7. Use the Python 3 standard library plus `psutil` and `PyYAML`. Ask before adding anything else.
8. Name the file `agent_<name>.py` and set `name = "<name>-agent"` on the class.

Extra notes for the file agent (watchdog, Louis):

- Watchdog calls its handler on its own thread. Put each event on a `queue.Queue` in the handler, and have `collect()` empty the queue and return the Events.
- Set `observed_at = time.time()` inside the handler when it fires, not in `collect()`, or MTTD picks up up to 2 seconds of fake delay.
- Hash on the `closed` event if your watchdog version has it, otherwise retry until the file size stops changing. Hashing on `created` can catch a half-written upload.
- Confirm Juice Shop's real upload folder on our VM. The Wazuh FIM rule must watch the same folder.

Your slice is done when:

- [ ] The EDR starts with your rule file in `rules/` and prints no errors
- [ ] Your attack script runs from Kali and appends to `attack_log.csv`
- [ ] Running your attack produces your alert in `alerts.jsonl` with the right `rule_id` and `mitre`
- [ ] Your 3+ malicious variants and 3+ benign look-alikes are in `tests/testcases.yaml`
- [ ] A full `score_eval.py` run shows your slice's TP, FP, TN and FN counts

- [ ] You saved a screenshot of the alert and your results for the progress report and final report
- [ ] Your rule file, scripts, test cases and screenshots are in the shared folder

Demo day and presentation

David runs the live demo. Louis, Thapanawat, Kikiola and Oliver split the slides between them at Monday's meeting. A suggested structure:

Slides	Covers
Problem and client	what the client needs, our scope, why MTTD is the focus
Design	the architecture and UML from this doc, MITRE mapping
Results	TP / FP / TN / FN per slice, MTTD vs Wazuh
Limitations and next steps	what we missed and why (POST bodies, XSS), future work
Live demo	2 to 3 attacks, run by David

Every slice owner sends their results row and one alert screenshot to whoever builds the Results slide by Wednesday night, so every slice is shown even if it is not demoed live.

Live demo plan:

- Only 2 or 3 attacks, as Yasas confirmed. We pick them from Wednesday's scoring run: detected every time, clear on screen, and ideally one Wazuh missed. RCE is a strong candidate, because Wazuh only saw it as a 500 error while our EDR sees the shell command itself.
- Run the chosen attacks in dry-run first, then repeat the last one with `--respond` to show the block or kill as the closing moment.
- Between rehearsals, a reset script restarts Juice Shop, flushes iptables and clears `alerts.jsonl`.

Morning-of checklist:

- [ ] Both VMs booted and Juice Shop loads from Kali
- [ ] Clocks match on both VMs (`date`)

- [ ] iptables flushed and `alerts.jsonl` cleared
- [ ] Terminal font enlarged for the room
- [ ] Backup demo video on a USB and in the shared folder